

# Lab09 – Helm

## Helm consume chart

In this exercise, you use Helm to install Kubernetes objects needed for the open source monitoring solution Prometheus (<https://prometheus.io/>). The easiest way to install Prometheus on top of Kubernetes is with the help of the prometheus-operator (<https://prometheus-operator.dev/>) Helm chart.

1. The Prometheus Helm charts reside in the artifact repository (<https://prometheus-community.github.io/helm-charts>). Add the repository to the list of known repositories accessible by Helm with the name `prometheus-community`.

```
vagrant@kubernetes-control-plane:~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
vagrant@kubernetes-control-plane:~$
```

2. Update to the latest information about charts from the respective chart repository.

```
vagrant@kubernetes-control-plane:~$ helm repo update prometheus-community
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ✌️Happy Helming!✌️
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$
```

3. Run the Helm command for listing available Helm charts and their versions. Identify the latest chart version for `kube-prometheus-stack`.

```
vagrant@kubernetes-control-plane:~$ helm search hub prometheus-community | grep kube
https://artifacthub.io/packages/helm/prometheus... 48.3.1 v0.66.0 kube-prometheus-stack collects Kubernetes manif...
https://artifacthub.io/packages/helm/prometheus... 5.10.1 2.9.2 Install kube-state-metrics to generate and expo...
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$
```

4. Install the the chart `kube-prometheus-stack`.

```
vagrant@kubernetes-control-plane:~$ helm install prometheus prometheus-community/kube-prometheus-stack
NAME: prometheus
LAST DEPLOYED: Sat Aug 12 15:19:52 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace default get pods -l "release=prometheus"

Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.
vagrant@kubernetes-control-plane:~$
```

```
vagrant@kubernetes-control-plane:~$ kubectl get deploy,svc,statefulset
NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/prometheus-grafana      1/1     1             1           11m
deployment.apps/prometheus-kube-prometheus-operator  1/1     1             1           11m
deployment.apps/prometheus-kube-state-metrics  1/1     1             1           11m

NAME                                     TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/alertmanager-operated           ClusterIP      None             <none>            9093/TCP,9094/TCP,9094/UDP  8m57s
service/kubernetes                      ClusterIP      10.96.0.1        <none>            443/TCP           28m
service/prometheus-grafana              ClusterIP      10.103.202.208   <none>            80/TCP           11m
service/prometheus-kube-prometheus-alertmanager  ClusterIP      10.104.212.198   <none>            9093/TCP,8080/TCP  11m
service/prometheus-kube-prometheus-operator  ClusterIP      10.98.35.185     <none>            443/TCP           11m
service/prometheus-kube-prometheus-prometheus  ClusterIP      10.99.100.174    <none>            9090/TCP,8080/TCP  11m
service/prometheus-kube-state-metrics      ClusterIP      10.103.47.194    <none>            8080/TCP           11m
service/prometheus-operated              ClusterIP      None             <none>            9090/TCP           8m57s
service/prometheus-prometheus-node-exporter  ClusterIP      10.101.174.8     <none>            9100/TCP           11m

NAME                                     READY   AGE
statefulset.apps/alertmanager-prometheus-kube-prometheus-alertmanager  1/1     8m57s
statefulset.apps/prometheus-kube-prometheus-prometheus                  1/1     8m57s
vagrant@kubernetes-control-plane:~$
```

## 5. List the installed Helm chart.

```
vagrant@kubernetes-control-plane:~$ helm list
NAME          NAMESPACE    REVISION    UPDATED                               STATUS    CHART          APP VERSION
prometheus    default       1           2023-08-12 15:19:52.160179782 +0000 UTC deployed  kube-prometheus-stack-48.3.1  v0.66.0
vagrant@kubernetes-control-plane:~$
```

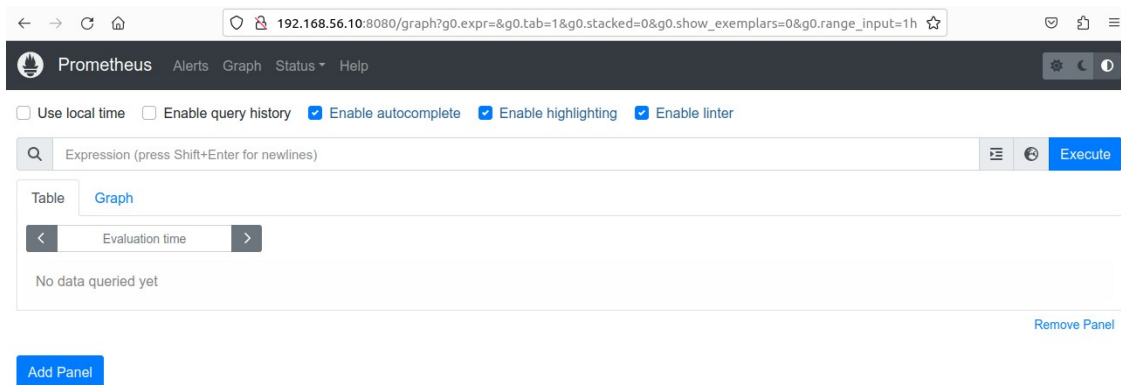
- List the Service named `prometheus-operated` created by the Helm chart. The object resides in the `default` namespace. This Service exposes the Prometheus dashboard on port 9090.

```
vagrant@kubernetes-control-plane:~$ kubectl get service prometheus-operated
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
prometheus-operated ClusterIP      None             <none>            9090/TCP          10m
vagrant@kubernetes-control-plane:~$
```

- Use the kubectl `port-forward` command to forward the local port 8080 to the port 9090 of the Service.

```
vagrant@kubernetes-control-plane:~$ kubectl port-forward --address 0.0.0.0 service/prometheus-operated 8080:9090
Forwarding from 0.0.0.0:8080 -> 9090
vagrant@kubernetes-control-plane:~$
```

## 8. Open a browser and bring up the Prometheus dashboard.



## 9. Stop port forwarding and uninstall the Helm chart.

```

vagrant@kubernetes-control-plane:~$ kubectl port-forward --address 0.0.0.0 service/prometheus-operated 8080:9090
Forwarding from 0.0.0.0:8080 -> 9090
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
^Cvagrant@kubernetes-control-plane:~$ helm uninstall prometheus
release "prometheus" uninstalled
vagrant@kubernetes-control-plane:~$

```

## Helm build chart

In this exercise, you will practice the implementation, packaging, and installation of a configurable custom Helm chart.

## 10. Create a new chart file named `Chart.yaml`. Define all mandatory attributes including the chart's API version, the name, and the version. Add the following key-value pairs: `API version: 1.0.0`, `Name: web-app`, and `Version: 2.5.4`.

```

vagrant@kubernetes-control-plane:~/lab$ vim Chart.yaml
vagrant@kubernetes-control-plane:~/lab$ cat Chart.yaml
apiVersion: 1.0.0
name: web-app
version: 2.5.4
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$

```

11. Create a new values file named `values.yaml`. It should contain the following key-value pairs: `service\_port: 80`, and `container\_port: 3000`.

```
vagrant@kubernetes-control-plane:~/lab$ vim values.yaml
vagrant@kubernetes-control-plane:~/lab$ cat values.yaml
service_port: 80
container_port: 3000
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

12. Create the template file `web-app-pod-template.yaml`. The YAML manifest defines a Pod named `hello-world` with the image `brahimhamdi/nodejs-hello-world:1.0.0`. The container port uses the placeholder `container\_port` from the `values.yaml` file.

```
vagrant@kubernetes-control-plane:~/lab$ vim templates/web-app-pod-template.yaml
vagrant@kubernetes-control-plane:~/lab$ cat templates/web-app-pod-template.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello-world
  namespace: {{ .Release.Namespace }}
  labels:
    app: hello-world
spec:
  containers:
    - image: brahimhamdi/nodejs-hello-world:1.0.0
      name: hello-world
      ports:
        - containerPort: {{ .Values.container_port }}
          protocol: TCP
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

13. Create the template file `web-app-service-template.yaml`. The YAML manifest defines a Service named `web-app-service` of type `ClusterIP`. The target port should use the placeholder `container\_port` from the `values.yaml` file, the port should use the placeholder `service\_port` from the `values.yaml` file.

```
vagrant@kubernetes-control-plane:~/lab$ vim templates/web-app-service-template.yaml
vagrant@kubernetes-control-plane:~/lab$ cat templates/web-app-service-template.yaml
apiVersion: v1
kind: Service
metadata:
  name: hello-world-service
  namespace: {{ .Release.Namespace }}
spec:
  type: ClusterIP
  selector:
    app: hello-world
  ports:
    - protocol: TCP
      port: {{ .Values.service_port }}
      targetPort: {{ .Values.container_port }}
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

14. Bundle the template files into a chart archive file by running the correct Helm command. What's the name of the file produced?

```
vagrant@kubernetes-control-plane:~/lab$ helm package .
Successfully packaged chart and saved it to: /home/vagrant/lab/web-app-2.5.4.tgz
vagrant@kubernetes-control-plane:~/lab$ ls
Chart.yaml  templates  values.yaml  web-app-2.5.4.tgz
vagrant@kubernetes-control-plane:~/lab$
```

15. Install the chart with the name `hello-world` to the namespace `app-stack` using the appropriate Helm command. Override the value of `service\_port` with the value `9090`.

```
vagrant@kubernetes-control-plane:~/lab$ helm install --namespace web-app --create-namespace --set service_port=9090 hello-world .
NAME: hello-world
LAST DEPLOYED: Sat Aug 12 15:57:40 2023
NAMESPACE: web-app
STATUS: deployed
REVISION: 1
TEST SUITE: None
vagrant@kubernetes-control-plane:~/lab$
```

16. Find the installed objects provided by the chart.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl get services,pods -n web-app
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/hello-world-service        ClusterIP   10.98.43.33    <none>         9090/TCP   46s

NAME          READY   STATUS             RESTARTS   AGE
pod/hello-world 0/1     ContainerCreating   0          46s
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```